



SUPERIOR UNIVERSITY, LAHORE

Gold Campus

DAIRY FARM MANAGEMENT SYSTEM

A Console-Based C++ Application

Project Report

Course: Programming Fundamentals (PF)

Semester 2 — Section BSSE-2A

Sr. No.	Student Name	Registration Number
1	Muhammad Haseeb	SU92-BSSEM-F25-001
2	Muhammad Asim	SU92-BSSEM-F25-054
3	Ali Hussan	SU92-BSSEM-F25-288
4	Ghulam Abbas	SU92-BSSEM-F25-008
5	Ali Raza	SU92-BSSEM-F25-016

Table of Contents

1. Introduction	3
2. Objectives	3
3. Tools and Technologies Used	3
4. Project Summary	3
5. Programming Concepts Used	4
6. System Modules	4
7. Input Validation and Authentication	5
8. File Handling and Data Persistence	5
9. Testing	5
10. Limitations and Future Enhancements	5
11. Conclusion	5

Introduction

The Dairy Farm Management System is a console-based application developed in C++ using the Dev-C++ IDE, as a Programming Fundamentals (PF) project for Semester 2. It provides a single, structured program for maintaining records of animals, milk production, feed consumption, animal health, and sales, in place of manual paper registers.

The project applies the core procedural programming concepts covered in the PF course — structures, arrays, functions, loops, and file handling — to build a complete, menu-driven record-management tool with persistent storage.

Objectives

- To design a modular, menu-driven system for managing dairy farm records.
- To apply structures for organizing related data of animals, milk, feed, health, and sales.
- To implement Add, View, Search, Update, and Delete operations for each record type.
- To validate user input and reject invalid or duplicate entries.
- To persist data between program runs using file handling.
- To restrict access to the system through a basic login mechanism.

Tools and Technologies Used

- Programming Language: C++
- IDE: Dev-C++
- Standard Libraries: iostream, fstream, string, limits
- Data Storage: Plain text file (dairy_data.txt) for saving and loading records

Project Summary

The table below summarizes the core characteristics of the system at a glance.

Aspect	Details
Programming Language	C++
IDE Used	Dev-C++
Application Type	Console-based application
File Handling	Yes — records saved to and loaded from a text file (dairy_data.txt)
CRUD Operations	Yes — Add, View, Search, Update, and Delete for every record type
Login Authentication	Yes — fixed username/password check before menu access
Input Validation	Yes — integers, decimals, non-blank strings, and DD-MM-YYYY dates are all validated

Programming Concepts Used

The following core C++ concepts, all verified in the source code, were applied in building the system:

Concept	Where Used in the Code
Structures (struct)	Animal, Milk, Feed, Health, and Sale structures group each record's related fields together.
Arrays	Fixed-size arrays (e.g., animals[MAX], milkRecords[MAX]) store all records of each type in memory.
Functions	Dedicated functions handle input validation, menu display, and the add/view/search/update/delete logic for every module.
Loops	for loops iterate over record arrays for display, search, and deletion; while/dowhile loops drive input validation and repeating menus.
File Handling	ifstream and ofstream are used in loadData() and saveData() to read and write all records to dairy_data.txt.

System Modules

After a successful login, the main menu gives access to five management modules, each offering the same set of operations: Add, View, Search, Update, Delete, and Back.

- Animal Management — animal ID, type (Cow, Buffalo, Goat), gender, and age.
- Milk Management — daily milk production linked to a specific animal, with quantity and date.
- Feed Management — feed type, quantity, and date of feeding.
- Health Management — health issues per animal and their status (Recovered, Sick, Treatment).
- Sales Management — sale transactions with item name, amount, and date.

Each module's core operations work as follows:

Operation	Description
Add	Prompts for the new record's details, rejects duplicate IDs, and — where relevant — verifies the referenced Animal ID exists.
View	Lists all records of that type, or reports that none exist.
Search	Finds and displays one record by its ID.
Update	Locates a record by ID and lets its editable fields be re-entered.
Delete	Removes a record by ID and shifts later entries left to close the gap.

Input Validation and Authentication

Reusable helper functions validate every value entered by the user: integers are checked for range and numeric type, decimals must be non-negative, text fields cannot be left blank, and dates must follow the DD-MM-YYYY format. Invalid entries are rejected with an error message, and the user is re-prompted until valid input is given.

Access to the system is protected by a login step: the program repeatedly asks for a username and password and only proceeds to the main menu once the correct credentials are entered.

File Handling and Data Persistence

All records are stored in a single text file, `dairy_data.txt`. The `saveData()` function writes the current count and details of every animal, milk, feed, health, and sales record to the file, and is called automatically after every add, update, or delete operation. The `loadData()` function reads this file back into memory when the program starts, so records are never lost between sessions.

Testing

Each module's Add, View, Search, Update, and Delete operations were tested with both valid and invalid input — duplicate IDs, non-existent Animal IDs, out-of-range menu choices, negative quantities, and badly formatted dates. In every case, the validation logic rejected the invalid input and re-prompted the user without the program crashing.

Limitations and Future Enhancements

- Fixed-size arrays (`MAX = 100`) cap the number of records per module; a dynamic container such as `std::vector` would remove this limit.
- Login credentials are hard-coded; a future version could store credentials in a file and support multiple users.
- Data is stored in a plain text file with no encryption; a database would improve data integrity and scalability.
- A graphical interface could replace the console interface for improved usability.

Conclusion

The Dairy Farm Management System meets the objectives set out for this Programming Fundamentals project. It demonstrates practical use of structures, arrays, functions, loops, and file handling to build a complete, working console application with persistent storage, input validation, and basic authentication — providing a solid foundation in structured C++ programming.